

ПОЛНОЕ РУКОВОДСТВО: СПИСКИ, МАТРИЦЫ И NUMPY В PYTHON## ЧАСТЬ 1: ГЛАВНЫЕ ПРАВИЛА ЧИСТОГО КОДА (BEST PRACTICES)

- * Не изобретайте велосипед: встроенные функции NumPy (np.zeros, np.diag, np.eye) написаны на языке C. Они работают в сотни раз быстрее обычных циклов Python.
- * Не ломайте встроенные имена: имена вроде list, sum, min, max, type зарезервированы. Используйте вместо них my_list, lst или diag_elements.
- * Избегайте «мертвого» кода: не создавайте переменные, которые не используются в дальнейшем. Python сам выделит память при присваивании.

ЧАСТЬ 2: РАБОТА С БАЗОВЫМИ СПИСКАМИ (LIST)

Список (list) – это изменяемая, упорядоченная коллекция элементов.

1. Способы создания списков

- * Пустой список: my_list = [] (рекомендуется) или my_list = list().
- * С готовыми элементами: numbers = [10, 20, 30, 40] (допускаются разные типы).
- * Автоматическая генерация:
- * Последовательность: digits = list(range(10)) -> [0, 1, ..., 9].
 - * Повторение: zeros = [0] * 5 -> [0, 0, 0, 0, 0].
 - * Генератор: squares = [x**2 for x in range(5)] -> [0, 1, 4, 9, 16].

2. Важная особенность типов данных (Проблема с np.int64)

При переносе элементов из NumPy в обычный список при выводе может появиться техническая обертка int64(...).

- * Решение: используйте генератор для очистки типа данных:

```
clean_list = [int(x) for x in your_list]
print(clean_list) # Выведет чистый список [1, 2, 3]
...
```

ЧАСТЬ 3: СОЗДАНИЕ И ОБЪЯВЛЕНИЕ МАТРИЦ

В Python нет встроенного типа данных "матрица". Они реализуются двумя путями:

Способ А: Вложенные списки (Чистый Python)

Подходят для простых задач и малых объемов данных.

- * Вручную:

```
matrix = [
    [1, 2, 3],
    [4, 5, 6]
]
...
```

- * Через генератор списков:

```
# Создание матрицы 3x3, заполненной нулями
matrix = [[0 for j in range(3)] for i in range(3)]
...
```

* КРИТИЧЕСКАЯ ОШИБКА: никогда не создавайте матрицу через умножение списков вроде `[[0]*3]*3`. Python создаст три ссылки на одну и ту же строку. Изменив один элемент, вы измените весь столбец!

Способ Б: Динамическое заполнение пользователем

Если размеры и элементы заранее неизвестны, матрицу заполняют в цикле `input()`:

```
rows = int(input("Введите количество строк: "))cols = int(input("Введите количество столбцов: "))matrix = []for i in range(rows):
    row = []
    for j in range(cols):
        element = int(input(f"Введите элемент [{i}][{j}]: "))
        row.append(element)
    matrix.append(row)
...
```

Способ В: Инициализация через NumPy (Рекомендуемый для вычислений)

* **Матрица нулей (`np.zeros`)**:

```
```python
import numpy as np
Передаем размер в виде кортежа (строки, столбцы)
matrix_zeros = np.zeros((3, 3))
```
```

* **Единичная матрица (`np.eye`)**:

квадратная матрица с единицами по главной диагонали.

```
```python
identity_matrix = np.eye(4) # Создает матрицу 4x4
```
```

ЧАСТЬ 4: ОПЕРАЦИИ С МАТРИЦАМИ (ТЕОРИЯ И КОД)

1. Правила работы с индексами

Синтаксис `matrix[row][col] = value` работает, но с ограничением. Нельзя обратиться к индексу, которого еще нет в памяти:

```
```python# ОШИБКА: IndexErrormatrix = []
```

```
matrix[0][0] = 5
```

```
ПРАВИЛЬНО (Сначала инициализация):matrix = [[0 for _ in range(3)] for _ in range(3)]
```

```
matrix[0][0] = 5 # Работает!
```

```
```
```

2. Нахождение размера матрицы

Способ получения размерности (строк, столбцов) строго зависит от типа данных:

* **Для списков**:

```
rows = len(matrix), cols = len(matrix[0]).
```

* **Для NumPy**:

```
свойство .shape возвращает кортеж (rows, cols).
```

```
```python
rows, cols = A.shape
```
```

Вызов `.shape` у обычного списка вызовет `AttributeError`.

3. Сложение матриц

Складывать можно только матрицы ****одинакового**** размера. Элементы складываются попарно.

```
```pythonmatrix_sum = A + B
```
```

4. Умножение матриц: Различайте операторы!

В NumPy есть критическая разница между математическим и поэлементным

перемножением.

* ****Умножение на скаляр (число)****: каждый элемент умножается на это число. Размер не меняется.

```
```python
scalar_mult = A * 5
```
```

* ****Поэлементное умножение матриц****: оператор `\*` (Hadamard product) умножает ячейки, стоящие на одинаковых позициях.

```
```python
poelementno = A * B
```
```

* ****Классическое матричное умножение****: оператор `@` или функция `np.dot()` выполняет умножение «строка на столбец».

```
```python
result = A @ B # Или np.dot(A, B)
```
```

5. Транспонирование

Строки матрицы становятся её столбцами, а размер MxN превращается в NxM. В NumPy выполняется через свойство `.T`:

```
```python
A_transposed = A.T
```
```

6. Главная диагональ и след матрицы

* ****Главная диагональ****: элементы из левого верхнего в правый нижний угол с индексами `[i][i]`. Извлекается функцией `np.diag(A)`.

* ****След матрицы (Trace)****: сумма всех элементов её главной диагонали $\text{Tr}(A)$. Вычисляется через `np.trace(A)`.

ЧАСТЬ 5: ФОРМАТИРОВАНИЕ И ВЫВОД ДАННЫХ

1. Форматированные F-строки

Буква `f` перед кавычками `print(f"...")` позволяет вставлять переменные и вычисления прямо внутрь текста в фигурных скобках `{}`:

```
```python
r, c = 2, 3
print(f"Матрица {r}x{c} содержит {r * c} элементов.")
```
```

2. Красивый вывод матриц без лишних скобок

По умолчанию `print()` выводит матрицы со всеми вложенными скобками. Сделать вывод аккуратным можно тремя способами:

* ****Способ 1: Построчный вывод без скобок (универсальный)****

```
```python
for row in matrix:
 print(*row)
```
```

* ****Способ 2: Удаление только внешних скобок****

```
```python
output = str(matrix).strip('[]')
print(output)
```
```

* ****Способ 3: Глобальная настройка вывода NumPy****

Используется для больших матриц или чисел с плавающей точкой:

```
```python
```

# 3 знака после запятой, без научной нотации (e+01), вывод без сокращений в троеточие (...)

```
np.set_printoptions(precision=3, suppress=True, threshold=np.inf)
``
```